

Vellore Institute of Technology

Cloud Architecture Design

ShopSphere

A Multi-Database E-Commerce Platform on AWS

Amazon RDS • Amazon Aurora • Amazon DynamoDB

Solution Proposal – Assignment Set-E

Member	Registration No.	Role
Sudeep Kumar Sahu	24BCT0160	Relational & Data Lead
Shrijay Sinha	24BCE2998	Application, NoSQL & Cloud Lead

Submitted to: **Prof. Sudhakar P**

Abstract

We propose **ShopSphere**, a working online shop deployed on Amazon Web Services in which customers register and log in, browse products, manage a cart, place orders, pay and track deliveries. Rather than forcing all data into one database, ShopSphere places each kind of data in the AWS database whose model matches its access pattern: **Amazon RDS for PostgreSQL** stores customers, products, orders and payments with full ACID guarantees; **Amazon Aurora PostgreSQL** hosts the same relational schema during a simulated festival sale, separating write traffic (orders) from read traffic (browsing) across instances in different Availability Zones; and **Amazon DynamoDB** keeps shopping carts, login sessions and user preferences as key-value documents. A runtime switch moves the application from RDS to Aurora without redeployment, and every screen displays which database and which instance served it, so each architectural claim can be verified live in the AWS Management Console. The design targets near-zero cost (a few dollars of AWS Free Tier credits) and is planned for a two-member team with clearly separated ownership.

Contents

1	Problem Statement	2
2	Objectives	2
3	Proposed Solution Overview	2
4	System Architecture	3
5	Database Design	4
5.1	Relational model (RDS and Aurora)	4
5.2	Checkout as one ACID transaction	4
5.3	Festival sale on Aurora	4
5.4	NoSQL model (DynamoDB)	5
6	Comparison of RDS, Aurora and DynamoDB	6
7	Security Design	6
8	Cost Analysis and Optimisation	7
9	Demonstration Plan	7
10	Team Roles and Contribution	8
11	Timeline	8
12	Risks and Mitigation	8
13	Expected Outcomes and Future Scope	9
14	Conclusion	9

1 Problem Statement

An e-commerce application must support six customer capabilities: *register and log in*, *browse products*, *add products to a cart*, *place orders*, *make payments* and *track orders*. The underlying data is not uniform:

- **Customers** → **Orders** → **Order Items** → **Products** form a chain of one-to-many relationships involving money and inventory. Integrity (no order without a customer, no payment without an order, no negative stock) is mandatory.
- During a **festival sale**, traffic can rise from about 1,000 to more than 100,000 concurrent users, and the increase is dominated by *reads* (browsing), while *writes* (orders) must remain correct and available.
- The **shopping cart**, **sessions** and **preferences** are small per-customer documents read and written on almost every click, always by a single key, and never joined with other data.

The assignment asks us to implement this use case on AWS, demonstrate that it works, and compare Amazon RDS, Amazon Aurora and Amazon DynamoDB.

2 Objectives

- O1** Deliver all six customer capabilities as a working web application on AWS.
- O2** Use **RDS** as the relational system of record and run the SQL queries given in the brief, unchanged, against application-generated data.
- O3** Demonstrate **Aurora**'s advantages for the festival scenario: writer/reader separation, read scaling, serverless capacity scaling and automatic failover.
- O4** Use **DynamoDB** for the cart (keyed by customer id, exactly the document shape in the brief), sessions (with automatic expiry) and preferences.
- O5** Produce an evidence-based comparison of the three services.
- O6** Keep AWS spending to a few dollars of Free Tier credits and delete all resources after the demonstration.

3 Proposed Solution Overview

ShopSphere is a single web application (React front end served by a Node.js/Express API) running on one small Amazon EC2 instance. The API talks to three data stores and routes every operation explicitly. Table 1 maps each customer capability to its data store.

Capability	Implementation	Data store
Register / log in	bcrypt-hashed password in customers ; session item with TTL; JWT cookie carries only the session id	RDS/Aurora + DynamoDB
Browse products	paginated search, filter, sort	RDS / Aurora reader
Cart	one document per customer, optimistic locking via a version attribute	DynamoDB
Place order	single ACID transaction with row locks on stock	RDS / Aurora writer
Payment	deterministic mock gateway, idempotency key, transactional status update	RDS / Aurora writer
Track order	status timeline from order_status_history	RDS / Aurora writer

Table 1: Capability-to-database mapping

Design idea that makes the demonstration verifiable

Every API response carries headers naming the data source, role and instance (for example `Aurora · reader · shopisphere-aurora-reader-1 · 4ms`). The UI shows these as coloured badges. An evaluator can therefore watch browsing traffic go to the Aurora reader and orders go to the writer, then confirm the same instances in the console.

4 System Architecture

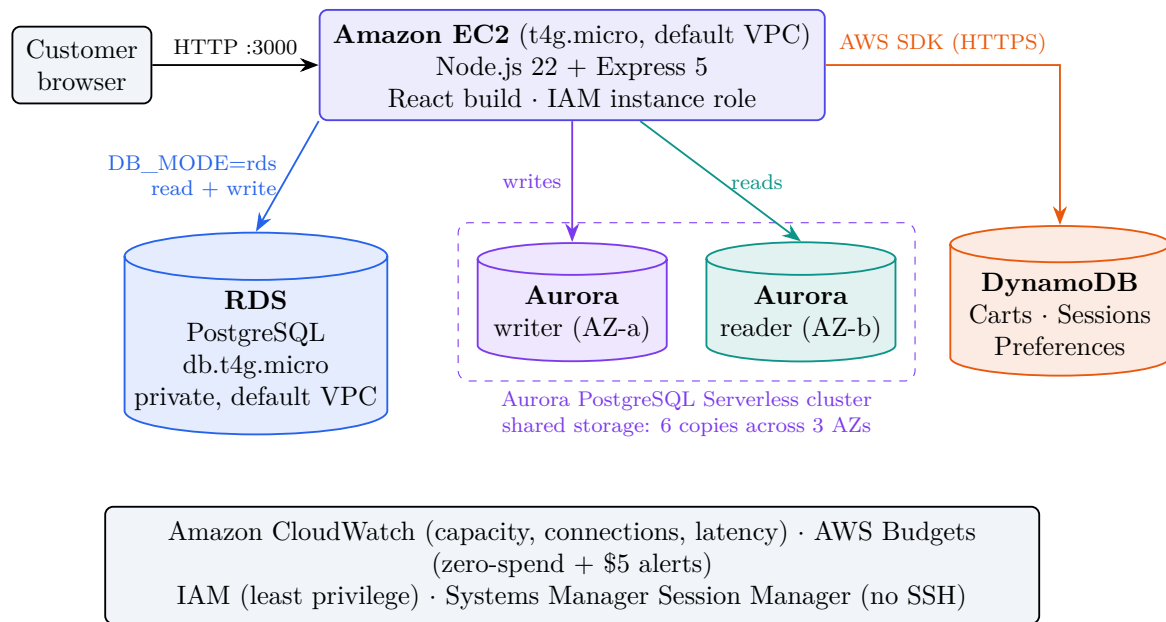


Figure 1: ShopSphere deployment architecture

Components.

- **EC2 application server** in the default VPC’s public subnet, managed by pm2. Shell access through AWS Systems Manager Session Manager (no SSH port open). AWS credentials come from an IAM instance role; no access keys are stored on the server.
- **Amazon RDS for PostgreSQL**: single-AZ db.t4g.micro, not publicly accessible; its security group accepts port 5432 only from the application’s security group. Represents normal-traffic operation.
- **Amazon Aurora PostgreSQL (serverless)**: one writer and one reader placed in different Availability Zones, capacity range 0–2 ACU, created with Aurora’s express configuration (the option available on the AWS Free plan), which exposes the cluster through an IAM-authenticated internet access gateway. Represents festival-sale operation.
- **Amazon DynamoDB**: three on-demand tables (ShopSphere_Carts, ShopSphere_Sessions with Time-to-Live, ShopSphere_UserPreferences).

Read/write routing. All SQL calls go through `db.read()` or `db.write()`. In RDS mode both resolve to the single instance; in Aurora mode **read** uses the reader endpoint and **write** the cluster (writer) endpoint. Catalog browsing and reports are reads; registration, checkout, payment and a customer’s own order history use the writer so that a customer always sees what they just wrote (read-your-writes).

5 Database Design

5.1 Relational model (RDS and Aurora)

The same PostgreSQL schema is used on RDS and Aurora. Customer identifiers start at 101 and product identifiers follow the brief (P100, P200, ...) so that the queries and documents in the assignment work unmodified.

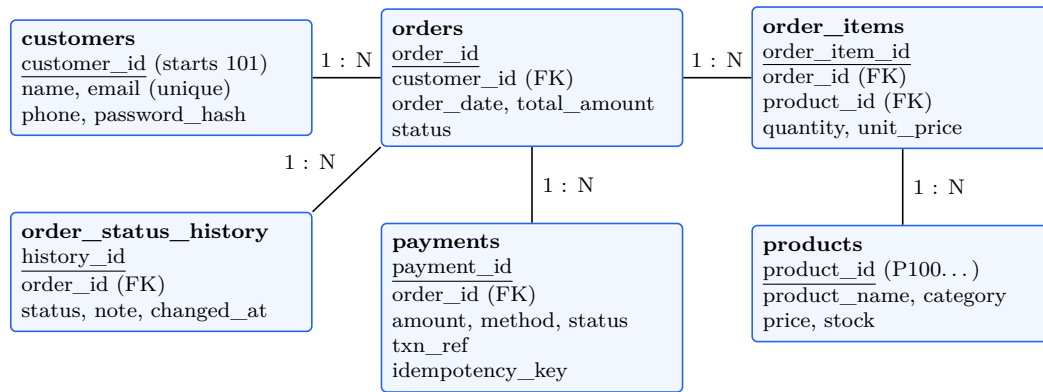


Figure 2: Entity-relationship model

Integrity is enforced by the database itself: primary and foreign keys, `UNIQUE(email)`, `CHECK (stock >= 0)`, `CHECK` constraints on status values, `NUMERIC` types for money and a unique idempotency key on payments.

5.2 Checkout as one ACID transaction

```

BEGIN;
SELECT product_id, price, stock FROM products
WHERE product_id = ANY($1) ORDER BY product_id FOR UPDATE; -- lock rows in fixed order
-- application verifies stock >= quantity for every line, else ROLLBACK
INSERT INTO orders (customer_id, total_amount, status) VALUES ($2, $3, 'PLACED') RETURNING order_id
;
INSERT INTO order_items (order_id, product_id, quantity, unit_price) VALUES (...);
UPDATE products SET stock = stock - $q WHERE product_id = $p; -- per line
INSERT INTO order_status_history (order_id, status) VALUES ($id, 'PLACED');
COMMIT; -- only now is the DynamoDB cart deleted
  
```

If any product lacks stock, the whole transaction rolls back: no order, no items, no stock change. This is demonstrated live with a low-stock product.

5.3 Festival sale on Aurora

- **Read scaling:** browsing (the bulk of festival traffic) is served by the reader; the writer handles only orders and payments. Aurora replicas share the cluster storage volume, so adding a reader does not copy data and replica lag stays low.
- **Elastic capacity:** each serverless instance scales in Aurora Capacity Units between 0 and 2 ACU and pauses when idle, so cost follows traffic.
- **Availability:** writer and reader sit in different Availability Zones; a manual failover promotes the reader, the cluster endpoint follows the new writer, and the application reconnects without a restart.

- **Migration:** the RDS database is copied to Aurora with `pg_dump/psql`; because both are PostgreSQL, no schema or query changes are required.

5.4 NoSQL model (DynamoDB)

Table	Partition key	TTL	Access pattern
ShopSphere_Carts	customer_id	–	GetItem / conditional PutItem / DeleteItem by customer
ShopSphere_Sessions	session_id	expires_at	PutItem at login, GetItem per request, DeleteItem at logout
ShopSphere_UserPreferences	customer_id	–	GetItem / UpdateItem (theme, categories, recently viewed)

Table 2: DynamoDB tables (on-demand capacity)

The cart item for customer 101 is exactly the document given in the brief, plus a version number used for optimistic concurrency control:

```
{ "customer_id": "101",
  "items": [ { "product_id": "P100", "quantity": 2 },
              { "product_id": "P200", "quantity": 1 } ],
  "version": 7 }
```

A cart is fetched with a single `GetItem` on `customer_id = "101"`: no SQL, no joins and no table scans. Session items expire automatically through DynamoDB Time-to-Live, and the application also checks the expiry timestamp because TTL deletion is not instantaneous.

6 Comparison of RDS, Aurora and DynamoDB

Feature	Amazon RDS	Amazon Aurora	Amazon DynamoDB
Database type	Relational (managed MySQL, PostgreSQL, MariaDB, Oracle, SQL Server)	Relational, cloud-native distributed storage (MySQL/PostgreSQL compatible)	NoSQL key-value and document
SQL	Yes, full SQL	Yes, full SQL	No SQL engine; key-based API plus PartiQL (SQL-like, single table)
JOINS	Yes	Yes	No; data is modelled per access pattern (denormalised)
Flexible schema	No; fixed columns, changed with <code>ALTER TABLE</code> (JSONB columns help)	No; same as RDS	Yes; only key attributes are fixed
Serverless options	No; an instance size is chosen	Yes; Aurora Serverless scales in ACUs and can pause at zero	Yes; fully serverless, on-demand capacity
Very large scale	Moderate; vertical scaling and asynchronous read replicas	High; up to 15 low-lag readers on shared storage replicated six ways across three AZs	Very high; virtually unlimited throughput and storage with consistent millisecond latency
Transactions	Full ACID, multi-table	Full ACID, multi-table	ACID transactions across up to 100 items (<code>TransactWriteItems</code>)
High availability	Optional Multi-AZ standby (extra cost)	Fast failover by promoting a reader	Built in, multi-AZ by default
Cost model	Instance hours + storage; cheapest for small steady loads	ACU-hours (serverless) or instance hours, plus storage and I/O	Per request (on-demand) or provisioned capacity, plus storage
Best for	Customer and order management at normal traffic	The same relational workload at festival scale: read-heavy spikes and high availability	Shopping carts, sessions and preferences: single-key, high-frequency access

Table 3: Feature comparison (completed table from the brief)

Evidence in the demonstration. JOINS and transactions are shown with the brief’s SQL and the checkout rollback on RDS; read scaling, serverless capacity and failover are shown on the Aurora cluster and CloudWatch; flexible schema and key-based access are shown with the cart document and PartiQL in the DynamoDB console. An *Insights* page reports measured median latencies for a DynamoDB `GetItem`, an Aurora reader query and a writer query, giving the comparison a quantitative element.

7 Security Design

- **Network:** RDS is private and reachable only from the application security group; the EC2 instance exposes only the application port; no SSH port is opened (Session Manager is used instead).
- **Identity:** the application uses an IAM instance role restricted to the three ShopSphere tables and to `rds-db:connect` for one database user on the Aurora cluster. Aurora connections use short-lived IAM authentication tokens instead of stored passwords.
- **Data in transit and at rest:** TLS for all database connections; storage encryption enabled on RDS, Aurora and DynamoDB.

- **Application:** parameterised SQL only, input validation, bcrypt password hashing, httpOnly session cookies with server-side revocation, rate limiting on login, a least-privilege database role, and no full card numbers stored.

8 Cost Analysis and Optimisation

New AWS accounts on the Free plan receive promotional credits, and Aurora PostgreSQL serverless is available on that plan within fixed limits (up to 4 ACUs and 1 GiB storage per cluster). We therefore expect **no out-of-pocket cost**. Development is done locally with Docker (PostgreSQL and DynamoDB Local), so AWS resources run for only about three days.

Resource	Configuration	Approx. 3-day cost
EC2 + public IPv4 + 10 GB gp3	t4g.micro	\$1.00
RDS for PostgreSQL	db.t4g.micro, Single-AZ, 20 GB	\$1.40
Aurora PostgreSQL serverless	writer + reader, 0–2 ACU, auto-pause	\$1–2
DynamoDB	3 on-demand tables	<\$0.05
CloudWatch, Budgets	basic metrics, 1 dashboard, 2 budgets	\$0
Total (paid from credits)		\$3–6

Table 4: Estimated consumption (approximate on-demand rates; verified in the console before creation)

Cost controls: zero-spend and monthly budgets with e-mail alerts; no NAT gateway, load balancer, Elastic IP or Multi-AZ RDS; Aurora pauses at zero capacity when idle; resources tagged **Project=ShopSphere** and stopped every evening; complete teardown immediately after recording, verified with Tag Editor and the billing console.

9 Demonstration Plan

The video (target 16 minutes, within the 10–20 minute limit) shows both members on camera alongside the AWS Management Console.

#	Time	Presenter	Content
1	0:00–1:00	Both	Introductions, problem, solution in one sentence
2	1:00–2:30	Shrijay Sinha	Architecture walk-through, cost guardrails (Budgets)
3	2:30–5:00	Shrijay Sinha	Register/login (session in DynamoDB), browse, cart document and PartiQL
4	5:00–8:00	Sudeep Kumar Sahu	RDS configuration, brief’s SQL queries, live order, rollback on low stock
5	8:00–9:15	Shrijay Sinha	Declined then successful payment, order tracking timeline
6	9:15–12:45	Sudeep Kumar Sahu	Aurora cluster, read-only reader proof, RDS→Aurora switch, festival load test with CloudWatch
7	12:45–13:45	Sudeep Kumar Sahu	Manual failover and recovery
8	13:45–15:15	Shrijay Sinha	Measured latencies and the comparison table
9	15:15–16:00	Both	Credits used, teardown, contributions, conclusion

Table 5: Video outline

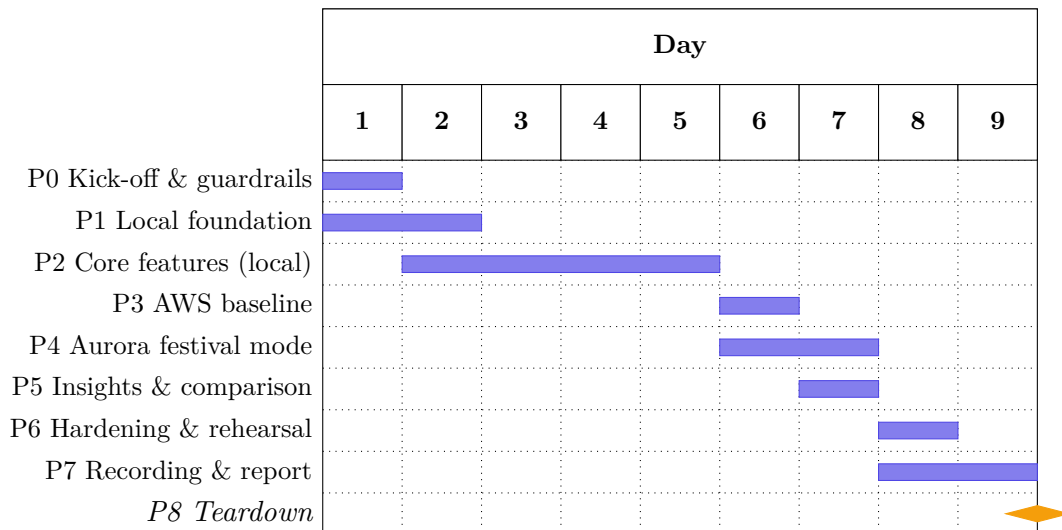
10 Team Roles and Contribution

Area	Sudeep Kumar Sahu (Relational & Data Lead)	Shrijay Sinha (Application, NoSQL & Cloud Lead)
AWS services	RDS, Aurora cluster and reader, migration, failover drill	DynamoDB, EC2, IAM role and policies, security groups, Budgets, CloudWatch
Backend	Connection pools and read/write routing, IAM token auth, products, checkout transaction, payments, insights API	Authentication and sessions, cart with optimistic locking, preferences, middleware and data-source headers
Frontend	Admin API integration	All pages, components and design system
Documentation	Architecture and database design, this proposal	Requirements, UI design, AWS setup guide, demo script
Video	Segments 4, 6, 7 (≈ 8 min)	Segments 2, 3, 5, 8 (≈ 8 min)

Table 6: Division of work

Both members review each other’s pull requests, so each understands the full system and can answer questions on any part.

11 Timeline



12 Risks and Mitigation

Risk	Mitigation
Credits consumed by forgotten resources	Budgets with alerts, nightly stop routine, tagged resources, teardown checklist
Aurora resumes slowly from zero capacity during recording	Minimum capacity raised to 0.5 ACU shortly before recording
IAM database authentication misconfiguration	Connection tested from CloudShell first; fallback to console-based SQL demonstration
Small instance runs out of memory during build	Swap file configured at launch
Video exceeds 20 minutes	Timed rehearsals and a prepared cut list

Table 7: Risk register

13 Expected Outcomes and Future Scope

Outcomes: a deployed shop covering all six capabilities; live proof of relational integrity on RDS, read/write separation, elastic capacity and failover on Aurora, and key-value access on DynamoDB; a completed, evidence-backed comparison; and a cost record showing the whole exercise ran on a few dollars of credits.

Future scope: Amazon Cognito for identity, a real payment gateway, Amazon CloudFront and S3 for static assets, DynamoDB Streams to publish order events, Aurora reader auto scaling, and infrastructure as code (AWS CDK) for repeatable environments.

14 Conclusion

ShopSphere shows that the question is not which AWS database is best, but which is best *for each part of the data*. Relational integrity belongs in RDS; the same relational workload under festival-scale, read-heavy traffic benefits from Aurora’s shared storage, readers and serverless scaling; and per-customer cart and session documents are served most simply and scalably by DynamoDB. The proposed implementation makes each of these choices visible and verifiable in a single working application, at negligible cost, with a clear and balanced division of work between the two team members.

References

- [1] Amazon RDS User Guide. <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/>
- [2] Amazon Aurora User Guide – Aurora Serverless, endpoints, failover. <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/>
- [3] Amazon Aurora – Create with express configuration. https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_GettingStartedAurora.AuroraPostgreSQL.ExpressConfig.html
- [4] Amazon Aurora on the AWS Free Tier. <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-free-tier.html>
- [5] Amazon DynamoDB Developer Guide – core components, TTL, transactions, PartiQL. <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/>
- [6] AWS Free Tier. <https://aws.amazon.com/free/>
- [7] IAM database authentication for Aurora. <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/UsingWithRDS.IAMDBAuth.html>